Ingestion Worker Deep Dive — Family Music Scrobbler PWA

Ingestion Worker Deep Dive - Family Music Scrobbler PWA

A comprehensive technical deep dive into the ingestion subsystem responsible for collecting Spotify listening history. This document explains architecture, scheduling, token security, deduplication logic, error handling, performance considerations, and operational behaviour.

📌 1. Purpose of the Ingestion Worker

The ingestion worker is responsible for:

- Polling Spotify's user-read-recently-played endpoint
- Deduplicating scrobbles using spotify_play_id
- Storing plays in monthly PostgreSQL partitions
- Managing encrypted refresh tokens
- Handling rate limits and backoff
- Maintaining ingestion continuity across restarts

It runs **inside the FastAPI backend container** using **APScheduler**.

🧱 2. Architectural Overview

Components

```
backend/app/workers/
  scheduler.py
  ingestion.py
  token_manager.py
  backoff.py
```

Data Flow

```
APScheduler → ingestion.py → Spotify API → dedupe → PostgreSQL → ingestion_state
```

Key Concepts

- **Polling interval:** 5-10 minutes per user
- **State tracking:** ingestion_state.last_polled
- **Token encryption:** AES key stored in Docker secrets

- **Partitioning:** scrobbles stored in monthly partitions

🔄 3. Scheduling Logic (APScheduler)

Scheduler Configuration

- Runs inside FastAPI startup event
- Uses BackgroundScheduler
- Interval job per user

Example

```
scheduler.add_job(run_ingestion, 'interval', minutes=poll_interval)
```

Why APScheduler?

- Works inside Python runtime
- Survives container restarts
- Easy to manage multiple users

🔐 4. Token Security & Refresh Lifecycle

Storage

Refresh tokens are stored encrypted:

```
spotify_tokens.refresh_token_enc (BYTEA)
```

Encrypted using AES key from:

```
/run/secrets/refresh_token_key
```

Refresh Flow

1. Decrypt refresh token
2. Request new access token
3. Store updated encrypted refresh token
4. Retry ingestion

Security Guarantees

- No plaintext tokens stored
- AES key never leaves container
- Tokens rotated automatically

🎧 5. Spotify API Polling Logic

Endpoint

```
GET https://api.spotify.com/v1/me/player/recently-played
```

Steps

1. Fetch recent plays
2. Normalize response
3. Deduplicate using spotify_play_id
4. Insert new scrobbles
5. Update last_polled

Normalized Scrobble

```
{
  "spotify_play_id": "string",
  "track_id": "uuid",
  "played_at": "2026-09-09T07:20:00Z"
}
```

🧹 6. Deduplication Strategy

Constraint

```
UNIQUE(user_id, spotify_play_id)
```

Why this works

- Spotify guarantees unique play IDs
- Prevents double ingestion
- Allows safe re-polling

Insert Logic

```
INSERT ... ON CONFLICT DO NOTHING
```

🗄️ 7. Partitioning Strategy

Scrobbles stored in monthly partitions:

```
scrobbles_2026_09
scrobbles_2026_10
```

Benefits

- Fast queries
- Reduced index bloat
- Easy archival

Partition Creation

- Automated via migration script
- Worker does not create partitions

⚠️ 8. Rate Limiting & Backoff

Spotify Rate Limits

429 responses indicate rate limiting.

Backoff Strategy

- Exponential backoff
- Retry after delay
- Log backoff events

Example

```
Retry in 30 seconds (attempt 2)
```

🧪 9. Error Handling

Categories

- Network errors
- Token refresh failures
- Spotify API errors
- Database errors

Behaviour

- Log error
- Retry with backoff
- Do not crash scheduler

Fatal Errors

Only fatal if:

- Token invalid and cannot be refreshed
- User revoked Spotify access

📊 10. Logging & Observability

Worker Logs

Include:

- Polling events
- New scrobbles count
- Backoff events
- Token refresh events

Health Endpoint

```
GET /healthz
```

Shows worker status.

Ingestion Status Endpoint

```
GET /ingestion/status
```

Shows:

- last_polled
- poll_interval
- worker status

🧭 11. Performance Considerations

Efficient Inserts

- Bulk inserts for scrobbles
- ON CONFLICT dedupe

Partitioning

- Keeps ingestion fast even with millions of rows

Token Refresh

- Cached access tokens reduce API calls

🛠️ 12. Troubleshooting

Ingestion Stopped

Check:

- Worker logs
- /ingestion/status
- Token refresh failures
- Spotify API availability

Duplicate Scrobbles

Check:

- Unique constraint
- spotify_play_id correctness

Missing Scrobbles

Check:

- Time zone handling
- Partition boundaries

🏁 13. Summary

The ingestion worker is a resilient, secure, and scalable subsystem that:

- Polls Spotify reliably
- Deduplicates scrobbles
- Stores data efficiently
- Handles rate limits gracefully
- Survives restarts

It is the backbone of the Family Music Scrobbler PWA's long-term analytics and historical tracking.